Convolution as a local operation

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Image-pattern Network Investigation

THE CONNECTED WEEK

Locate the window. Reuse the kernel. Keep the raw sum.

Compute complete local filters while preserving orientation,shape and boundary assumptions.

Keep one habit: Complete four-window arithmetic → labelled weighted patches → pictured shared bias → full stride/padding/orientation probes → independent new image/kernel.

MAIN LESSON · SECTION 1

Which local window produced this output?

> Keep kernel orientation, stride, padding, bias and activation explicit.

Learning goals

I can compute a local weighted sum at each valid image position.

I can explain shared weights and predict **output** dimensions.

I can check how **stride** boundaries and activation affect the result.

Retrieve the image-axis contract

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Image tensors and channels. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Debugging shapes and gradients. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from **Training** and validation curves. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

Prerequisite and starting response

- Why must an image schema record axis order as well as shape?

> **My starting explanation**
>
>
>
>
>
>
>
>

End goal

By the end, I can explain convolution as a local operation using a traced example and a stated limitation.

▼ **Earlier wording, alternatives and picture transcripts**

Your learning target

Remember before reading

This week you will practise:

Compute a local weighted sum at each valid image position

Explain shared weights and predict **output** dimensions

Check how **stride** boundaries and activation affect the result

Retrieve what you already know

This week’s end goal

MAIN LESSON · SECTION 2

Multiply matching entries and add every local product

> Reuse the same fixed-orientation kernel at every allowed position.

Complete image, kernel and boundary contract

Use a fictional single-channel image with rows [1, 2, 0], [0, 1, 3] and [2, 0, 1]. Coordinates are zero-based row then column. Our **kernel** has rows [1, -1] and [0, 1].

At each position, multiply matching entries of a 2 by 2 window and the **kernel**, then add the four products. Use bias 0, **stride** 1 and no **padding**.

At the top-left window, the sum is 1 times 1 plus 2 times -1 plus 0 times 0 plus 1 times 1 = 0. Slide the same **kernel** right, then down. Only use windows that fit entirely inside the image; this is the valid boundary rule.

All four valid windows and products

Window start	Four products	Sum
[0, 0]	1, -2, 0, 1	0
[0, 1]	2, 0, 0, 3	5
[1, 0]	0, -1, 0, 0	-1

Window start	Four products	Sum
[1, 1]	1, -3, 0, 1	-1

Name the declared fixed-orientation convention

The **feature map** has rows [0, 5] and [-1, -1]. This neural-network convention keeps the **kernel** orientation fixed; mathematically it is cross-correlation. Mathematical convolution reverses the **kernel** along both spatial axes.

We use the fixed-orientation convention throughout, so every hand calculation and program agrees.

Name given and missing facts

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

See each window and separate weights from repeated operations

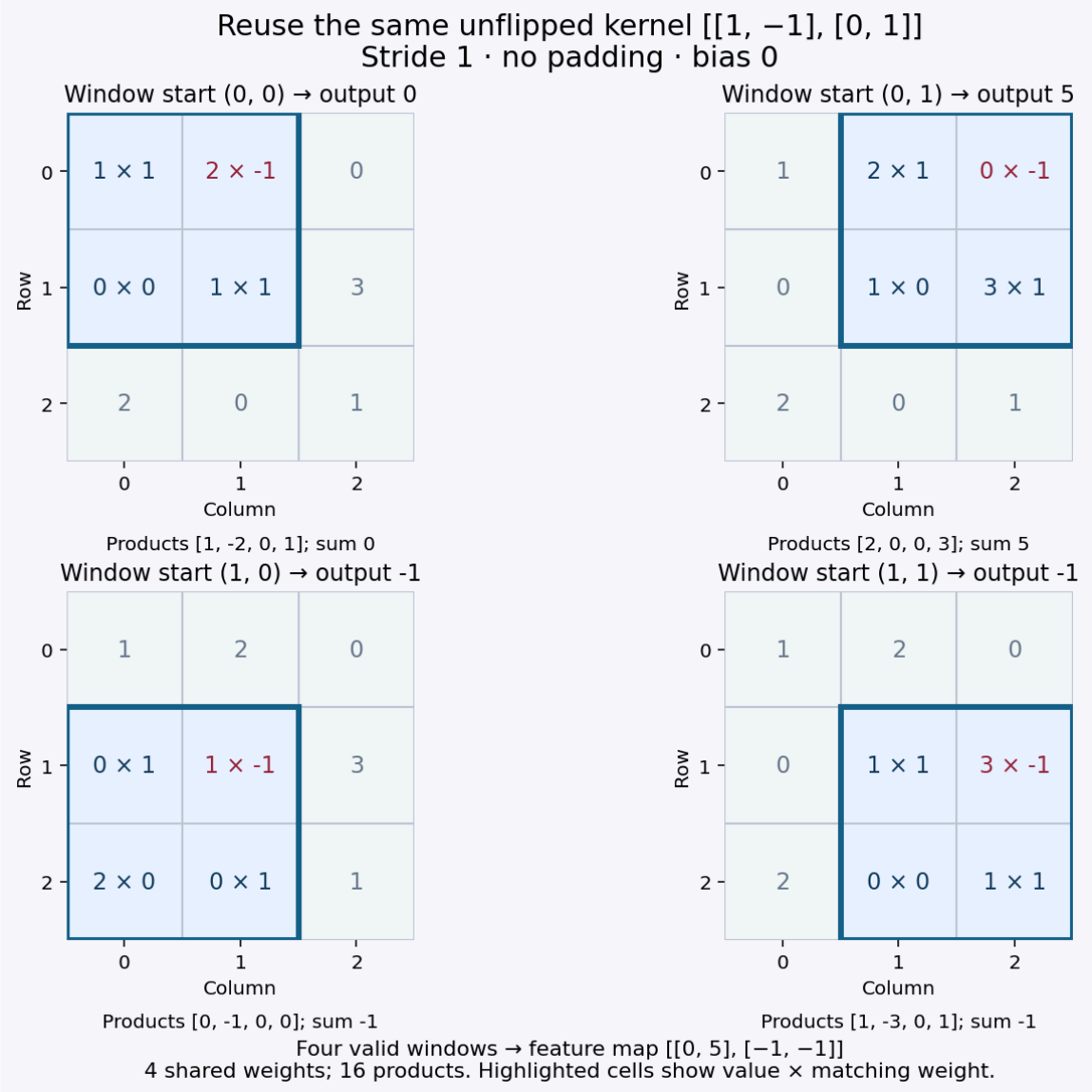
> A reused parameter can participate in many products.

Function scope and input assumptions

The following function assumes nonempty rectangular numeric arrays and a positive integer **stride**. It rejects a **kernel** larger than the image.

It computes one **input** channel, one **output** map, zero bias and no **padding**; no activation is applied inside the function.

Complete four-window product graphic



Shapes, parameter slots and operation counts

Main local operation	Complete count / meaning
Input / kernel / output	3×3 image, 2×2 kernel, 2×2 feature map
Allowed starts with stride 1	Rows0/1 and columns 0/1:four windows
Declared shared kernel values	Four weights reused at all four starts
Products in this direct algorithm	Four per window×four windows=16, including products with weight 0
Add one shared bias	Five parameter slots; add that bias once per output
Activation	No activation inside local_filter; apply a declared ReLU afterward if needed
Status	Hand-chosen arithmetic kernel, not a fitted recognition model or measured hardware-operation benchmark

The **kernel** weights are reused at all four positions. There are four **kernel** parameters, not sixteen separate weights. If we add one shared bias for this **output** map, there are five parameters.

The number of multiply operations grows with the number of windows even though the parameter count stays fixed.

Complete runnable fixed-orientation filter

```
def local_filter(image, kernel, stride=1):
    h, w = len(image), len(image[0])
    kh, kw = len(kernel), len(kernel[0])
    if kh > h or kw > w:
        raise ValueError("Kernel does not fit")
    out = []
    for r in range(0, h - kh + 1, stride):
        row = []
        for c in range(0, w - kw + 1, stride):
            value = sum(image[r+i][c+j] * kernel[i][j]
                        for i in range(kh) for j in range(kw))
            row.append(value)
        out.append(row)
```

```
    return out

x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]
assert local_filter(x, k) == [[0, 5], [-1, -1]]
assert local_filter(x, k, 2) == [[0]]
```

Connect the highlighted patch to its output coordinate

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Change one declared condition and trace its consequence

> Output dimensions and values depend on the complete operation contract.

Current parameter-versus-product task

- How many shared **kernel** weights does the lesson use? How many products are computed over all four valid windows? What changes if one shared bias is added?

Keep the procedure fixed. Explain what changed in the inputs and result.

Predict the pictured shared bias

- Change: Add shared bias 2 after each patch sum.

▼ Check after predicting

Prediction: **Feature map** becomes [[-2,-2]].

Reason: Bias adds 2 to each **output**; it does not change patch products or **output** shape.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Track one shared bias through every window

> The pictured input and kernel differ from the main four-window example.

Complete pictured arrays and separate scope

Draw the exact patch-by-patch arithmetic of a local image operation.

Image rows [1, 2, 3],[4, 5, 6]; **kernel** rows [1, 0],[0,-1]. Use unflipped **kernel**, no **padding**, **stride** 1, bias 0.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

The picture uses a different 2×3 image[[1, 2, 3], [4, 5, 6]]and unflipped kernel[[1, 0], [0, −1]], stride 1, no padding, bias 0. Its two windows give 1−5=−4 and2−6=−4, map[[-4, −4]]. Adding only shared bias 2 gives[[-2, −2]]without changing products, weight sharing or1×2 shape. These negative raw responses are allowed. The main 3×3 image instead gives[[0, 5], [−1, −1]].

Keep the image and kernel beside each calculation.

Trace both patch products and the raw map



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. First patch	1×1+2×0+4×0+5×(-1)=-4.
2. Second patch	2×1+3×0+5×0+6×(-1)=-4.
3. Feature map	Output shape 1×2 and map [[-4,-4]]; the same kernel is reused at each position.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

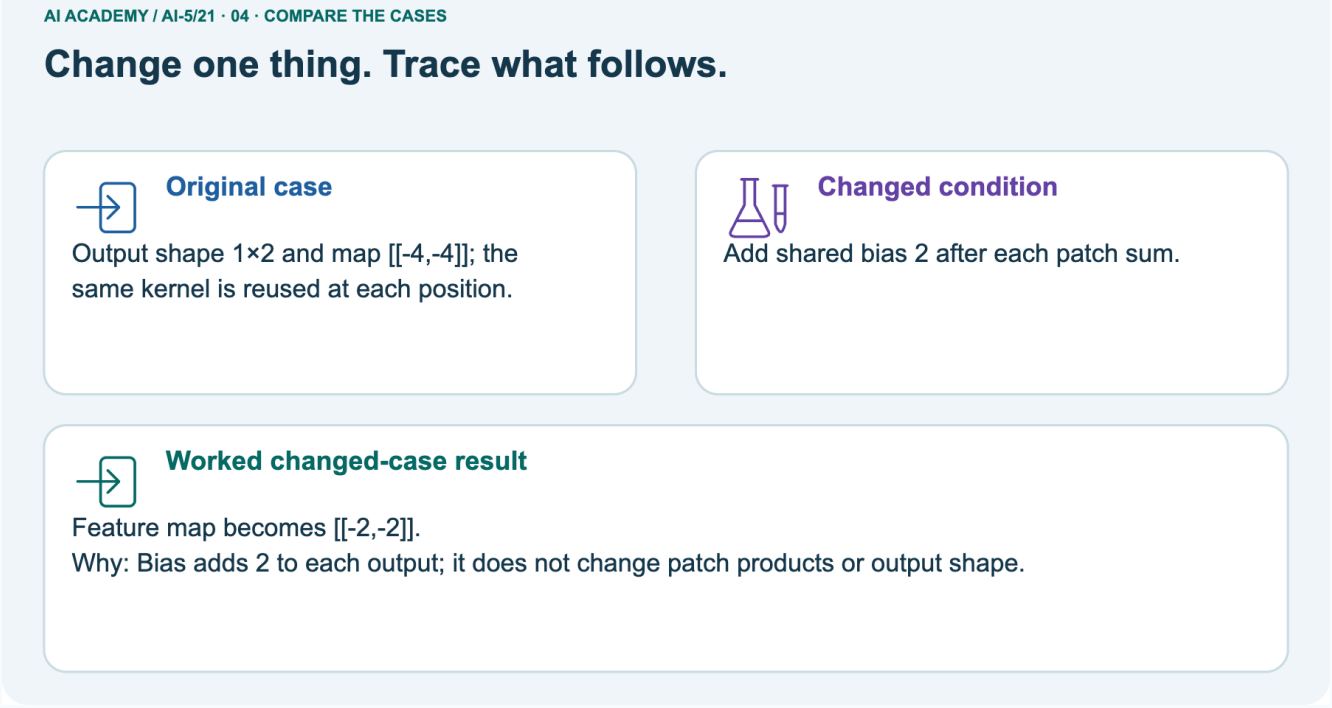
A local **kernel** combines the values in one small image window with a fixed set of weights. Slide that same **kernel** to the next valid window and repeat the products.

Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped **kernel**; adding bias changes each window result without changing which pixels were used.

Predict the shared bias2addition

- Change one thing: Add shared bias 2 after each patch sum. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

Explain orientation changes and recognition limits

- A local filter detects a numerical pattern; one **feature map** does not establish recognition or learned visual understanding.

- Read the labelled picture: Why is flipping the **kernel** an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Try the main one-pixel change

One shared kernel produces four local sums

WINDOW START	KERNEL ROWS	OUTPUT VALUE
Row 0 column 0	[1 -1] and [0 1]	0
Row 0 column 1	[1 -1] and [0 1]	5
Row 1 column 0	[1 -1] and [0 1]	-1
Row 1 column 1	[1 -1] and [0 1]	-1

Read windows in row order. Each output summarizes one neighbourhood. With stride 1, adjacent windows overlap, but their output values occupy separate cells.

Change only input[0][0] from 1 to 4. Which valid windows include it? Compute the changed feature map and explain why the other three outputs stay the same.

▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Convolution as a local operation

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

AI ACADEMY / AI-5/21

Convolution as a local operation

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. First patch

1 x 1+2 x 0+4 x 0+5 x (-1)=-4.



2. Second patch

2 x 1+3 x 0+5 x 0+6 x (-1)=-4.



3. Feature map

Output shape 1 x 2 and map [[-4,-4]]; the same kernel is reused at each position.

CHANGE ONE THING

Add shared bias 2 after each patch sum.

Explain the changed result using the matching lesson.

Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Count legal start positions before using a shape formula

> Padding supplies a boundary assumption;activation is a separate step.

Complete stride, padding and ReLU explanation

For no **padding** and a **kernel** that fits, **output** height is floor((H - Kh) / s) + 1 and width is floor((W - Kw) / s) + 1, where s is the positive integer **stride**. Floor means round down.

Count allowed start positions before using the formula. With a 3 by 3 image, 2 by 2 **kernel** and **stride** 2, the only row and column start is 0, so the **output** is 1 by 1.

Padding changes which windows exist and what they contain. Adding one border of zeros on every side changes this 3 by 3 **input** to 5 by 5. At **stride** 1 with our 2 by 2 **kernel**, **output** shape becomes 4 by 4.

Its top-left window is [[0, 0], [0, 1]] and its **output** is 1. Zero **padding** is a chosen boundary assumption, not a measured black region outside the picture.

If ReLU is applied after the unpadded sums, the **feature map** becomes [[0, 5], [0, 0]]. Negative responses are valid weighted sums before activation; they are not necessarily a software error.

Keep raw sums available when debugging so ReLU does not hide a mistake.

All one-factor shape and value checks

One-factor main-image variant	Complete output / boundary meaning
Original stride 1, no padding, bias 0	[[0, 5], [−1, −1]], shape 2×2
Stride2, same image/kernel	Only start(0, 0); map[[0]], shape 1×1
One zero border per side, stride 1	Padded5×5 input→4×4 map; top-left window[[0, 0], [0, 1]]gives 1
ReLU after original raw map	[[0, 5], [0, 0]]; raw negative sums remain valid debugging evidence
Reverse both axes of kernel	[[1, 0], [−1, 1]]is a changed convention; top-left result 2 rather than 0

One-factor main-image variant	Complete output / boundary meaning
Change only input[0][0]1→4	[[3, 5], [−1, −1]]; only first valid window contains that corner

Try the stride2and zero-border cases

- For the lesson image and **kernel**, give the **output** shape with no **padding** and **stride** 2. Then give the shape with one zero border on each side and **stride** 1, and calculate that padded map's top-left **output**.

> **State one repair and the checks you would rerun.**

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise local products and shared counts with less help

> Name which inputs and settings each result uses.

Current task and response

- Use the lesson image and **kernel** at window start [0, 1]. List the window rows, four products and **output** before activation.

Change only input[0][0] from 1 to 4. Which valid windows include it? Compute the changed feature map and explain why the other three outputs stay the same.

Explain your prediction to a partner before comparing results.

Supported W2record

Use workbook W2 for the connected example “Convolution as a local operation”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Individual misconception check

- Use the worked case for Convolution as a local operation. Proposed explanation: “Each window has its own set of learned weights”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Pictured fixed-orientation reasoning

- Why is flipping the **kernel** an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

> **My explanation using a pictured step and the given facts**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Reserve the independent new image/kernel

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Separate parameters and operations

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Run a complete window and boundary-condition audit

> Every array,setting and expected map is supplied.

Current count task and original local-filter function

How many shared **kernel** weights does the lesson use? How many products are computed over all four valid windows? What changes if one shared bias is added?

```
def local_filter(image, kernel, stride=1):
    h, w = len(image), len(image[0])
    kh, kw = len(kernel), len(kernel[0])
    if kh > h or kw > w:
        raise ValueError("Kernel does not fit")
    out = []
    for r in range(0, h - kh + 1, stride):
        row = []
        for c in range(0, w - kw + 1, stride):
            value = sum(image[r+i][c+j] * kernel[i][j]
                        for i in range(kh) for j in range(kw))
            row.append(value)
        out.append(row)
    return out

x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]
assert local_filter(x, k) == [[0, 5], [-1, -1]]
assert local_filter(x, k, 2) == [[0]]
```

Complete runnable one-factor probe program

```
def local_filter(image, kernel, stride=1):
    h, w = len(image), len(image[0])
    kh, kw = len(kernel), len(kernel[0])
    if kh > h or kw > w:
        raise ValueError("Kernel does not fit")
    out = []
    for r in range(0, h - kh + 1, stride):
        row = []
        for c in range(0, w - kw + 1, stride):
            value = sum(image[r+i][c+j] * kernel[i][j]
                        for i in range(kh) for j in range(kw))
            row.append(value)
        out.append(row)
    return out

x = [[1, 2, 0], [0, 1, 3], [2, 0, 1]]
k = [[1, -1], [0, 1]]
assert local_filter(x, k) == [[0, 5], [-1, -1]]
assert local_filter(x, k, 2) == [[0]]
# Complete declared one-factor probes, with original source function above.
changed = [row[:] for row in x]
changed[0][0] = 4
```

```
print('original', local_filter(x,k))
print('one input changed', local_filter(changed,k))
print('stride 2', local_filter(x,k,2))
padded = [[0]*5] + [[0]+row+[0] for row in x] + [[0]*5]
print('one zero border', local_filter(padded,k))
flipped = [row[::-1] for row in k[::-1]]
print('flipped kernel', flipped, 'map', local_filter(x,flipped))
raw_map = local_filter(x,k)
print('ReLU after raw sums', [[max(0,v) for v in row] for row in raw_map])
print('shared bias 2', [[v+2 for v in row] for row in raw_map])
print('4 shared weights, 16 products, 5 parameters if one shared bias is added')
# Nonempty rectangular numeric arrays and positive integer stride assumed.
```

Full padded map for independent checking

Complete one-zero-border map · Column0 · Column1 · Column2 · Column3

> **Complete one-zero-border map:** Row0
>
> **Column0:** 1
>
> **Column1:** 2
>
> **Column2:** 0
>
> **Column3:** 0

> **Complete one-zero-border map:** Row1
>
> **Column0:** -1
>
> **Column1:** 0
>
> **Column2:** 5
>
> **Column3:** 0

> **Complete one-zero-border map:** Row2
>
> **Column0:** 2
>
> **Column1:** -1
>
> **Column2:** -1

Column3: 3

Complete one-zero-border map: Row3

Column0: -2

Column1: 2

Column2: -1

Column3: 1

Normal,one-window,oversized,missing and orientation probes

Complete local-operation probe	Expected result / action
Normal main filter	Four sums 0/5/−1/−1in row-major 2×2 map
Kernel equals image dimensions	Exactly one valid window at(0, 0)under stride 1
Kernel larger than image	ValueError:Kernel does not fit
Missing/ragged numeric image or nonpositive stride	Outside nonempty rectangular numeric/positive-integer-stride contract; validate before calling
Shared bias 2on original main map	[[2, 7], [1, 1]]; four additions, one shared parameter
Zero padding at top-left	Assumed zeros create a new window; output 1, not an observed border pixel
Flipped kernel on main input	Map[[2, 4], [−2, 2]]; different from declared fixed orientation
Largest original response 5	A local weighted sum, not probability, recognition or learned visual understanding

Predict,filter,compare and retain raw maps

1. Predict all original windows before running or tracing local_filter. Then run or trace the complete probe program, changing one declared setting at a time. Save raw sums, output shapes and the full padded map; apply bias or ReLU only where declared. The function rejects an oversized kernel but assumes valid nonempty rectangular numeric arrays and positive integer stride; it is not a complete image validator.

   Count the sixteen products in the supplied direct algorithm separately from four shared weight slots. No training or real image recognition was performed by these hand-chosen arithmetic examples.

2. On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Keep the evidence ledger

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: filter a new image with a different kernel

> Use its own products,shared bias and separate final ReLU.

Complete new independent arrays and task

Use image rows [2, 1, 0], [1, 3, 2] and **kernel** rows [0, 1], [-1, 2]. Use no **padding**, **stride** 1 and bias 0. Give the **output** shape, both four-product sums and the **feature map**. Then add a shared bias of -5 and apply ReLU.

First attempt and support record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Filter a new image independently

Attempt this without hints

Use image rows [2, 1, 0], [1, 3, 2] and **kernel** rows [0, 1], [-1, 2]. Use no **padding**, **stride** 1 and bias 0. Give the **output** shape, both four-product sums and the **feature map**. Then add a shared bias of -5 and apply ReLU.

> **My independent procedure, result and limitation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

MAIN LESSON · SECTION 11

Keep the local-operation contract beside the feature map

> The project record should identify exactly which window and settings produced each result.

Project artifact and evidence table

Add a labelled convolution as a local operation **evidence** sheet to Image-pattern Network Investigation. Show how you can compute a local weighted sum at each valid image position.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

Actual run/trace and limits

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish with one local sum and one complete shape contract

> Explain the difference between repeated operations and shared parameters.

Retrieve and explain

- Close your notes. Explain this goal in your own words: Compute a local weighted sum at each valid image position

My deciding step, evidence and one remaining question

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use the mathematical words precisely**

REFERENCE · SECTION 5

Reference · Use the mathematical words precisely

> Connect each term to a value or step in the current example.

Vocabulary and your explanation

Word	Meaning
Kernel	A small array of weights applied to a local input neighbourhood
Feature map	The spatial array of outputs produced by applying a filter
Stride	The distance between successive window starting positions
Padding	Values added around an input boundary under a declared rule

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

Explain,repair and extend

- Use the lesson image and **kernel** at window start [0, 1]. List the window rows, four products and **output** before activation.
- For the lesson image and **kernel**, give the **output** shape with no **padding** and **stride** 2. Then give the shape with one zero border on each side and **stride** 1, and calculate that padded map's top-left **output**.
- A program reverses both axes of the lesson **kernel** to [[1, 0], [-1, 1]]. Compute the top-left **output**. Does it agree with our declared convention? Explain what the test checks.

Explain → apply → challenge

Explain which evidence supports your conclusion and which claim remains unproved.

▼ Reference · Check the representation and its limits

REFERENCE · SECTION 12

Reference · Check the representation and its limits

> Use after practice;reserve the new independent image/kernel with bias and ReLU for the first individual attempt.

Current explanation and response

- These **kernel** weights were chosen for arithmetic practice, not fitted from data. A response of 5 is neither a probability nor proof that an object was recognised.

  A trainable convolutional layer adjusts shared parameters using **training** examples. Additional channels require summing contributions across channels; that is beyond this single-channel calculation.

- Sharing weights applies the same local rule at each allowed position. It does not guarantee that a whole image classifier ignores object location, especially when **stride**s, boundaries or later layers change the result.

  Next week studies pooling and the spatial information it can discard.

- A program reverses both axes of the lesson **kernel** to [[1, 0], [-1, 1]]. Compute the top-left **output**. Does it agree with our declared convention? Explain what the test checks.

> **A teammate reports a 3 by 3 output for the unpadded lesson image and calls its largest number a probability. Identify the missing or incorrect assumptions and give the expected shape and meaning under our declared setup.**

My limited claim and a useful next check

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

Complete trace and evidence

Complete evidence record

One shared kernel produces four local sums

WINDOW START	KERNEL ROWS	OUTPUT VALUE
Row 0 column 0	[1 -1] and [0 1]	0
Row 0 column 1	[1 -1] and [0 1]	5
Row 1 column 0	[1 -1] and [0 1]	-1
Row 1 column 1	[1 -1] and [0 1]	-1

Read windows in row order. Each output summarizes one neighbourhood. With stride 1, adjacent windows overlap, but their output values occupy separate cells.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Reference · The earlier route checklist

REFERENCE · SECTION 15

Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

Targets,route and first prediction

Start with this question: Draw the exact patch-by-patch arithmetic of a local image operation.

By the end, I can	How I will show it
Compute a local weighted sum at each valid image position	Explain the deciding step, show a trace or test, and state one limit.
Explain shared weights and predict output dimensions	Explain the deciding step, show a trace or test, and state one limit.
Check how stride boundaries and activation affect the result	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Extra practice · Predict,check and repair

REFERENCE · SECTION 17

Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

First supported question and optional hint

- Draw the exact patch-by-patch arithmetic of a local image operation.
- Which labelled step matches this detail? **Output** shape 1×2 and map [[-4,-4]]; the same **kernel** is reused at each position.
- 1. **Feature map**
- 2. First patch
- 3. Second patch
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ Check after the first attempt

After your attempt, compare with the supported explanation: **Feature map Output** shape 1×2 and map [[-4,-4]]; the same **kernel** is reused at each position.

A local **kernel** combines the values in one small image window with a fixed set of weights. Slide that same **kernel** to the next valid window and repeat the products.

Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped **kernel**; adding bias changes each window result without changing which pixels were used.

Changed-case question and optional hint

- Changed-case practice: Add shared bias 2 after each patch sum. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. **Feature map** becomes [[-2,-2]].
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: **Feature map** becomes [[-2,-2]]. Bias adds 2 to each **output**; it does not change patch products or **output** shape.

Repair and retain the record

Review Image tensors and channels (ai-5/20). Goal: Read a pixel using declared row column and channel axes. Short practice: Where is the bottom green pixel in the first batched image?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review Debugging shapes and gradients (ai-5/18). Goal: Check dimension contracts before computing a layer. Short practice: What should a debugging record pair with every gradient value?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

Review **Training** and validation curves (ai-5/15). Goal: Compute comparable **training** and validation losses at saved checkpoints. Short practice: Can the original epoch 3 **training** loss 1 justify claiming best generalisation?

Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ Earlier wording, alternatives and picture transcripts

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

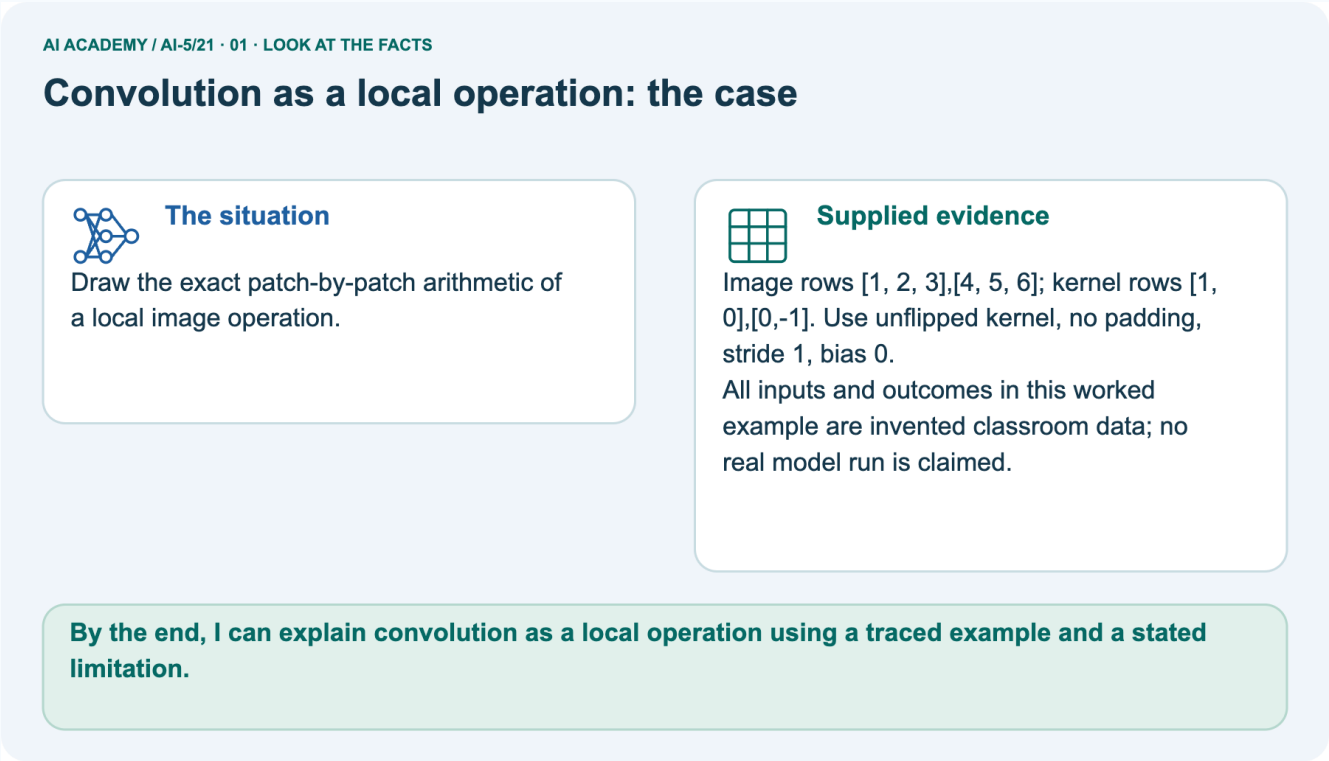
Keep your completed work

▼ Reference · Six-picture concept journey

Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

01. Convolution as a local operation: the case



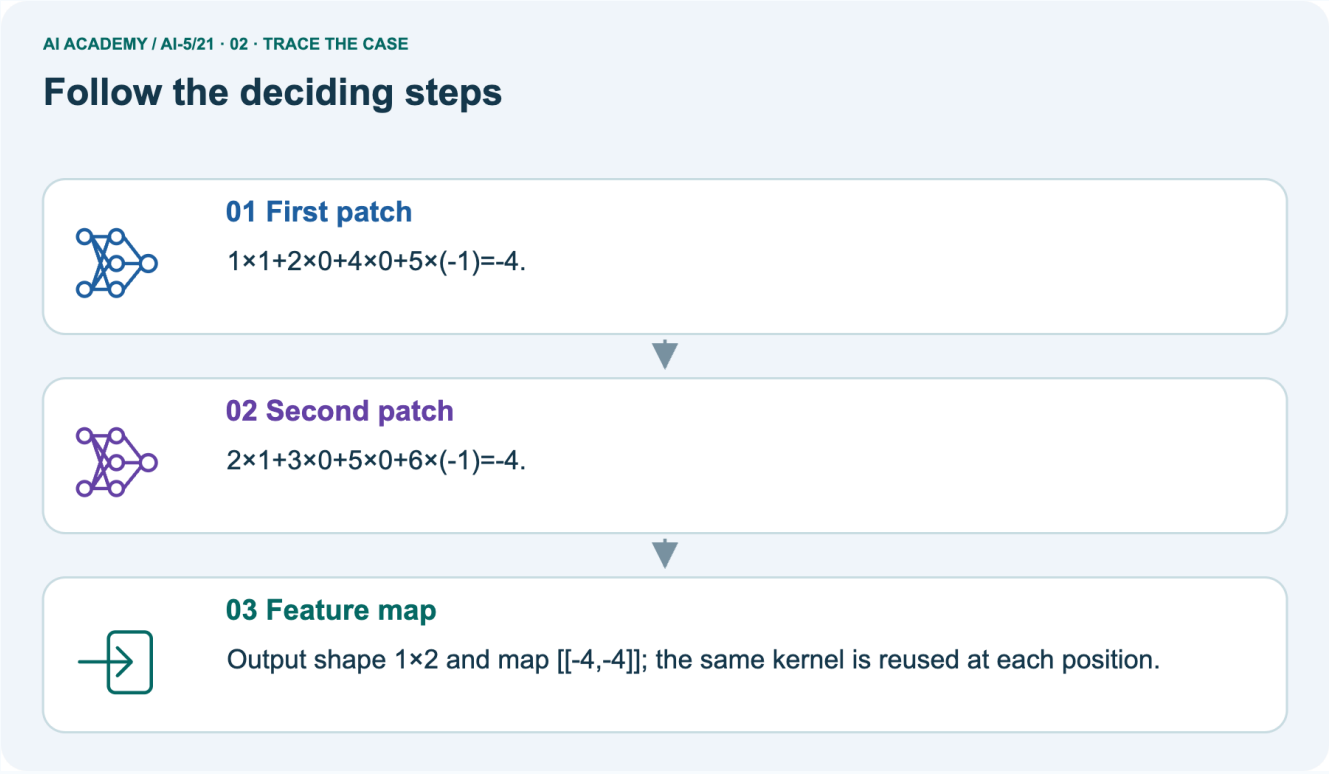
Follow the labelled picture with the matching explanation. Open the vector diagram

Draw the exact patch-by-patch arithmetic of a local image operation. Supplied facts: Image rows [1, 2, 3],[4, 5, 6]; **kernel** rows [1, 0],[0,-1]. Use unflipped **kernel**, no **padding**, **stride** 1, bias 0.

All **input**s and outcomes in this worked example are invented classroom data; no real model run is claimed.

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps



Follow the labelled picture with the matching explanation. Open the vector diagram

First patch: 1×1+2×0+4×0+5×(-1)=-4. → Second patch: 2×1+3×0+5×0+6×(-1)=-4. → **Feature map**: **Output** shape 1×2 and map [[-4,-4]]; the same **kernel** is reused at each position.

Point to each numbered stage and explain which recorded **input** it uses.

03. Image used by both local kernel windows

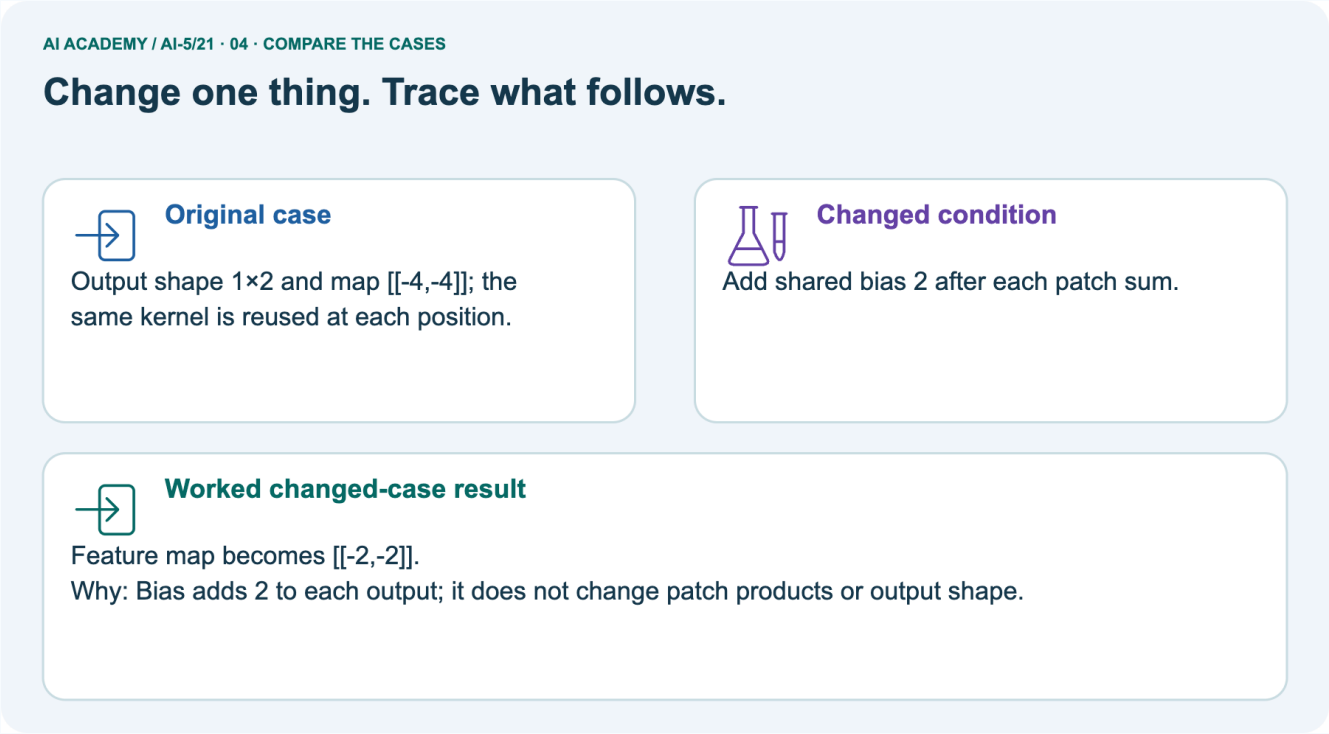


Follow the labelled picture with the matching explanation. Open the vector diagram

Image used by both local **kernel** windows. row 0: col 0 = 1, col 1 = 2, col 2 = 3. row 1: col 0 = 4, col 1 = 5, col 2 = 6.

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



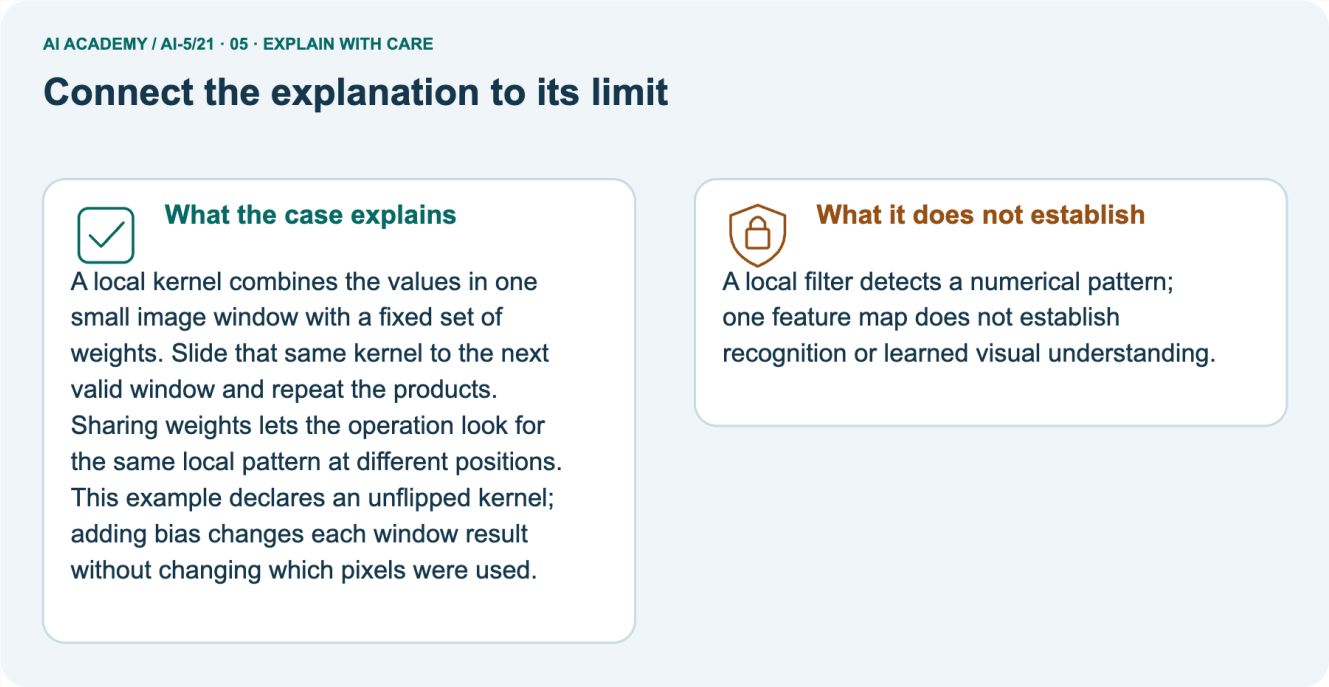
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: **Output** shape 1×2 and map [[-4,-4]]; the same **kernel** is reused at each position. Changed condition: Add shared bias 2 after each patch sum. Expected result: **Feature map** becomes [[-2,-2]].

Reason: Bias adds 2 to each **output**; it does not change patch products or **output** shape.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

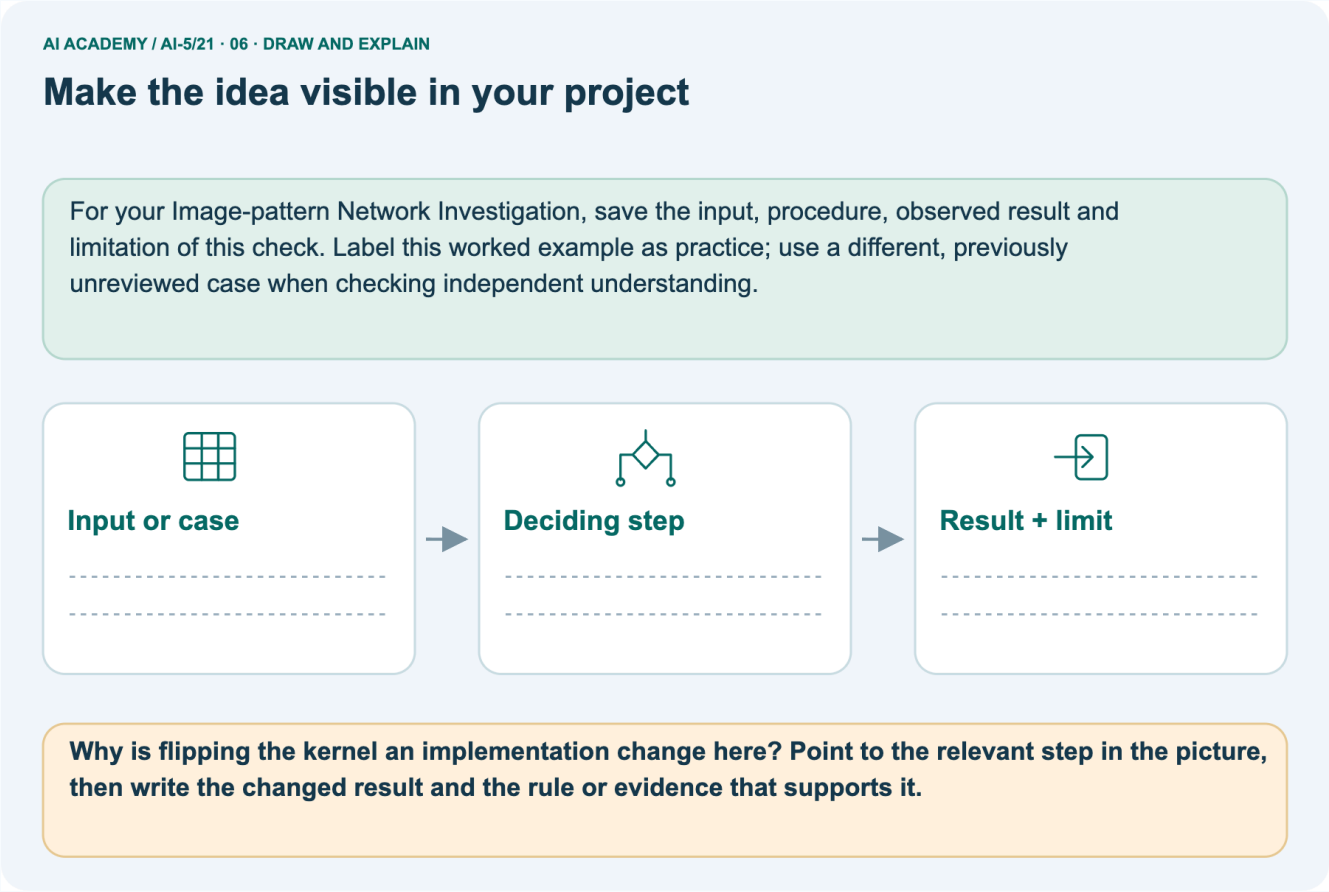
A local **kernel** combines the values in one small image window with a fixed set of weights. Slide that same **kernel** to the next valid window and repeat the products.

Sharing weights lets the operation look for the same local pattern at different positions. This example declares an unflipped **kernel**; adding bias changes each window result without changing which pixels were used.

Limit: A local filter detects a numerical pattern; one **feature map** does not establish recognition or learned visual understanding.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For your Image-pattern Network Investigation, save the **input**, procedure, observed result and limitation of this check.

Label this worked example as practice; use a different, previously unreviewed case when checking independent understanding. Draw your **input**, deciding step, result and limit.

Supported question: Why is flipping the **kernel** an implementation change here? Point to the relevant step in the picture, then write the changed result and the rule or **evidence** that supports it.

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Convolution as a local operation: the case

02. Follow the deciding steps

03. Image used by both local kernel windows

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.